

.NET 10 App Dev Hands-On Lab

MVC Lab 3 –Pipeline Configuration, Dependency Injection

This lab updates the HTTP pipeline, configuration, and dependency injection. Before starting this lab, you must have completed MVC Lab 2b.

Part 1: Update the Project File

- If you updated the tables as temporal tables (EF Core Lab 8), comment out the `IncludeAssets` tag for **EntityFrameworkCore.Design** in the `AutoLot.Mvc.csproj` file:

```
<PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="[10.0.*,11.0)">
<!--<IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>-->
<PrivateAssets>all</PrivateAssets>
</PackageReference>
```

Part 2: Add the GlobalUsings.cs File

- Create a new file named GlobalUsings.cs in the AutoLot.Mvc project and update the contents to the following:

```
global using AutoLot.Dal.EfStructures;
global using AutoLot.Dal.Extensions;
global using AutoLot.Dal.Initialization;
global using AutoLot.Dal.Repos;
global using AutoLot.Dal.Repos.Interfaces;
global using AutoLot.Mvc.Models;
global using AutoLot.Services.Logging.Configuration;
global using AutoLot.Services.Logging.Interfaces;
global using AutoLot.Services.Simple;
global using AutoLot.Services.Simple.Interfaces;
global using AutoLot.Services.ViewModels;
global using Microsoft.AspNetCore.Mvc;
global using Microsoft.AspNetCore.Mvc.Infrastructure;
global using Microsoft.EntityFrameworkCore;
global using Microsoft.Extensions.DependencyInjection.Extensions;
global using Microsoft.Extensions.Options;
global using Microsoft.EntityFrameworkCore.Diagnostics;
global using System.Diagnostics;
global using System.Text.Json.Serialization;
```

Part 3: Update the Program.cs Top Level Statements

Step 1: Add Logging

- Add Serilog to the WebApplicationBuilder and the logging interfaces to the DI container in Program.cs in the AutoLot.Mvc project:

```
var builder = WebApplication.CreateBuilder(args);
builder.ConfigureSerilog(builder.Configuration);
builder.Services.RegisterLoggingInterfaces();
```

Step 2: Update WebHost for CSS Isolation

- The CSS Isolation file is created in the development environment or when an app is published. To create the CSS file in other environments, update the web host to use static web assets:

```
builder.Services.RegisterLoggingInterfaces();
if (!builder.Environment.IsDevelopment())
{
    builder.WebHost.UseStaticWebAssets();
}
```

Step 3: Add Anti Forgery Token to the Entire Application

- Update the AddControllersWithViews call to the following (changes in bold):

```
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
})
```

Step 4: Add DI Validation

- Add the code to validate DI services and scopes on building the web app:

```
builder.Services
    .AddControllersWithViews(options =>
    {
        options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
    })
    .AddControllersAsServices()
    .AddViewComponentsAsServices()
    .AddTagHelpersAsServices();
builder.Host.UseDefaultServiceProvider(o =>
{
    o.ValidateOnBuild = true;
    o.ValidateScopes = true;
});
```

Step 5: Add the Repos and ApplicationDbContext

- Use the configuration manager to get the connection string and the extension method in `AutoLot.Dal` to add the context and repos:

```
var connectionString = builder.Configuration.GetConnectionString("AutoLot");
builder.Services.AddAutoLotDal(connectionString);
```

Step 6: Add the Remaining Services

- Add the keyed services into the DI container:

```
builder.Services.AddKeyedScoped<IService, SimpleServiceOne>(nameof(SimpleServiceOne));
builder.Services.AddKeyedScoped<IService, SimpleServiceTwo>(nameof(SimpleServiceTwo));
```

- Add the following code to populate the `DealerInfo` class from the configuration file:

```
builder.Services.Configure<DealerInfo>(builder.Configuration.GetSection(nameof(DealerInfo)));
```

- Add the `HttpContextAccessor`:

```
builder.Services.AddHttpContextAccessor();
```

Step 7: Call the Data_INITIALIZER

- In the section after `builder.Build()`, flip the `IsDevelopment` if block around, and add the `UseDeveloperExceptionPage` and the code to rebuild the database so the code looks like this:

```
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
    if (app.Configuration.GetValue<bool>("RebuildDataBase"))
    {
        using var scope = app.Services.CreateScope();
        var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
        SampleDataInitializer.ClearAndReseedDatabase(dbContext);
    }
}
else
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
```

Step 8: Update the Routing for Attribute Routing

- Back in `Program.cs` in the `AutoLot.Mvc` project, comment out the call to `MapControllerRoute` and add the `MapControllers` call:

```
app.MapControllers();
//app.MapControllerRoute(
//    name: "default",
//    pattern: "{controller=Home}/{action=Index}/{id?}")
//    .WithStaticAssets();
```

Part 4: Add WebOptimizer

This section shows how to use WebOptimizer for bundling, minification, and caching. The MapStaticFiles method (introduced in ASP.NET Core 9) and WebOptimizer do not work together yet, so it must be commented out.

Step 1: Add WebOptimizer to DI Container

- Update the Program.cs top-level statements by adding the following code after adding the services but before the call to builder.Build():

```
if (builder.Environment.IsDevelopment() || builder.Environment.IsEnvironment("Local"))
{
    builder.Services.AddWebOptimizer(false, false);
}
else
{
    builder.Services.AddWebOptimizer(options =>
    {
        //options.MinifyCssFiles(); //Minifies all CSS files
        options.MinifyCssFiles("css/**/*.css");
        //options.MinifyJsFiles(); //Minifies all JS files
        options.MinifyJsFiles("js/site.js");
        //options.MinifyJsFiles("js/**/*.js");
    });
}
var app = builder.Build();
```

Step 2: Add UseWebOptimizer and UseStaticFiles to the HTTP Pipeline

- Add in the calls to UseWebOptimizer and UseStaticFiles and comment out the MapStaticAssets code in Program.cs as shown here:

```
app.UseHttpsRedirection();
app.UseWebOptimizer();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
//app.MapStaticAssets();
app.MapControllers();
```

Step 3: Update _ViewImports to enable WebOptimizer Tag Helpers

- Update the _ViewImports.cshtml file to enable WebOptimizer tag helpers:

```
@using AutoLot.Mvc
@using AutoLot.Mvc.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
@addTagHelper *, WebOptimizer.Core
```

Part 5: Update the Home Controller

Step 1: Update the Controller and Action method routing

- Add the Controller level route to the HomeController (the commented-out route shows the equivalent route using a literal instead of a token):

```
[Route("[controller]/[action]")]
public class HomeController : Controller
```

- Add `HttpGet` attribute to all Get action methods:

```
[HttpGet]
public IActionResult Index()

[HttpGet]
public IActionResult Privacy()
```

- Add the default, controller only, and controller/action routes to the Index action method:

```
[Route("/")]
[Route("/[controller]")]
[Route("/[controller]/[action]")]
[HttpGet]
public IActionResult Index()
```

Step 2: Add a primary constructor that gets `IAppLogger` and test it

- Remove the constructor and the logger field if it exists. Add a primary constructor to the class and inject `IAppLogger` into the constructor:

```
public class HomeController(IAppLogger logger) : Controller
```

- Update the HomeController Index method to log an error:

```
public IActionResult Index()
{
    logger.LogAppError("Test error");
    return View();
}
```

- Run the application and make sure to launch a browser. Since the Index method is the default entry point for the application, just running the app should create an error file and an entry into the Serilog table. Once you have confirmed that logging works, comment out the error logging code:

```
//logger.LogAppError("Test error");
```

- Inject the DealerInfo OptionsSnapshot into the Index method and pass the Value to the View (the view will be updated in a later lab):

```
public IActionResult Index([FromServices]IOptionsSnapshot<DealerInfo> dealerOptionsSnapshot)
{
    return View(dealerOptionsSnapshot.Value);
}
```

- Inject the SimpleService into two new action methods and pass the message from the service to the SimpleService view (the view will be created in the next lab):

```
[HttpGet]
public IActionResult GetServiceOne(
    [FromKeyedServices(nameof(SimpleServiceOne))] ISimpleService service)
{
    return View("SimpleService",service.SayHello());
}
```

```
[HttpGet]
public IActionResult GetServiceTwo(
    [FromKeyedServices(nameof(SimpleServiceTwo))] ISimpleService service)
{
    return View("SimpleService",service.SayHello());
}
```

Summary

This lab added the necessary classes to the DI container and modified the application configuration.

Next steps

In the next part of this tutorial series, you will add support for client-side libraries, update the layout, and add GDPR Support.